

Relatório de Análise de Vulnerabilidades

Projeto: Sistema de Atendimento e Execução de Serviços — Oficina Mecânica (MVP, Fase 1) **Data:** 28/06/2026 **Ambiente:** Node 20.19.5, npm 10.8.2 · commit analisado `e578cfb` **Ferramentas:** `npm audit` (SCA — dependências) e SonarQube Community 26.6.0 (SAST — código)

1. Sumário executivo

A análise combinou duas frentes complementares:

- SCA** (*Software Composition Analysis*) com `npm audit` — procura vulnerabilidades conhecidas nas **dependências** (o `node_modules`).
- SAST** (*Static Application Security Testing*) com SonarQube — procura vulnerabilidades, *bugs* e *security hotspots* no **código que escrevemos**.

Resultado após a remediação aplicada (migração NestJS 11 + bcrypt 6):

Frente	Resultado
SonarQube (código)	Quality Gate: PASSED — 0 vulnerabilities, 0 security hotspots, 0 bugs
npm audit (produção)	2 moderadas, 0 altas, 0 críticas
npm audit (grafo completo)	20 moderadas, 0 altas, 0 críticas

A evolução das dependências em relação ao estado inicial do MVP:

	Antes (Nest 10)	Depois (Nest 11)
Grafo completo (prod + dev)	49 (10 altas)	20 (0 altas)
Apenas produção (runtime)	18 (6 altas)	2 (0 altas)

Conclusão: não há nenhuma vulnerabilidade **crítica ou alta**, nem no código nem nas dependências. As 20 remanescentes do `npm audit` são, na verdade, **um único advisory** (ver §3) — moderado e **não explorável** no contexto desta aplicação. Risco geral do MVP avaliado como **baixo**.

2. SAST — SonarQube (análise do código)

Análise estática do código-fonte (`src/`), com cobertura de testes importada do Jest.

Métrica	Valor	Rating
Vulnerabilities	0	A (Security)

Métrica	Valor	Rating
Security Hotspots	0	—
Bugs	0	A (Reliability)
Code Smells	39	A (Maintainability)
Cobertura	80,0%	—
Duplicação	0,0%	—
Linhas de código	5.691	—
Quality Gate	PASSED	—

Leitura: o código não introduz vulnerabilidades nem *security hotspots* próprios. Os 39 *code smells* são de manutenibilidade (rating A), não de segurança. A cobertura de 80,0% medida pelo Sonar abrange **todo** o `src/` (inclusive controllers, adapters e módulos); os domínios críticos (`dominio/` + `aplicacao/`) — foco do *threshold* de 80% do projeto — estão em ~98%, com o restante das linhas vindo das camadas de I/O (cobertas pelos testes e2e).

Como reproduzir: ver o **Anexo — Comandos executados** ao final (sobe o SonarQube via Docker, gera a cobertura e roda o scanner).

3. SCA — npm audit (análise das dependências)

`npm audit --omit=dev` → **2 moderadas (0 altas/críticas)** em produção. `npm audit` (completo) → **20 moderadas**.

3.1 As 20 são o mesmo advisory

Apesar do número, **todas convergem para uma única vulnerabilidade**: o `js-yaml <= 4.1.1` — *Quadratic-complexity DoS in merge key handling via repeated aliases* ([GHSA-h67p-54hq-rp68](https://nvd.nist.gov/vuln/detail/CVE-2020-35469)). O `npm audit` conta uma ocorrência para **cada pacote** que depende do `js-yaml` vulnerável:

- **2 em produção:** `@nestjs/swagger` → `js-yaml` (gera o YAML do OpenAPI).
- **18 em desenvolvimento:** a cadeia do `jest` (`@jest/*`, `jest-*`, `ts-jest`, `babel-jest`, `@istanbuljs/load-nyc-config` → `js-yaml`).

3.2 Por que não é explorável aqui

A vulnerabilidade é um **DoS ao fazer parsing de YAML não confiável** (com *merge keys* e *aliases* repetidos). Nesta aplicação:

- O `@nestjs/swagger` apenas **gera (dump)** o YAML do nosso próprio contrato OpenAPI — nunca faz *parse* de YAML vindo do usuário.

- A cadeia do `jest` roda **só em desenvolvimento/CI**, com YAML de configuração próprio, e **não entra na imagem Docker** de produção (é `devDependency`; o multi-stage build + o `.dockerignore` garantem que só o `dist/` e as deps de produção cheguem à imagem).

Não há, portanto, nenhum ponto onde YAML de terceiros seja parseado. Superfície de ataque real: nula.

3.3 Por que não corrigir agora

A correção do `js-yaml` só existe na versão **5.0.0** (mudança de *major*). Forçá-la via `overrides` traz risco de quebrar o `@nestjs/swagger` (que declara `js-yaml@^4`) sem nenhum ganho de segurança real (a vuln não é explorável). A decisão é **aguardar** o `@nestjs/swagger` adotar o `js-yaml@5` e atualizar então. Aceite de risco registrado.

4. Remediação aplicada

As vulnerabilidades **altas** do estado inicial foram efetivamente **eliminadas** com a atualização do stack (não é plano futuro — já está no código):

- **NestJS 10 → 11** (`@nestjs/*`) + **Express 4 → 5**: removeu as altas de `express / body-parser / qs`, `lodash` e `multer` (via `@nestjs/platform-express`).
- **bcrypt 5 → 6**: eliminou a cadeia `tar / @mapbox/node-pre-gyp` (file smuggling).
- **override de multer@2.2.0**: corrigiu o DoS do `multer` ([GHSA-72gw-mp4g-v24j](#) / [GHSA-3p4h-7m6x-2hcm](#)).
- **Ferramentas de dev** atualizadas (`@nestjs/cli 11` etc.): removeu as antigas vulnerabilidades de `webpack`, `tmp` e `inquirer`.

Validação da remediação: suíte de **200 testes unitários** (domínio + aplicação) reexecutada em 28/06/2026 com `npm run test:cov` — todos passando, sem regressões; o SonarQube importou a cobertura desse mesmo run (cobertura global 80,0%).

5. Recomendações de processo (boas práticas contínuas)

1. **npm audit no CI** — falhar o build em severidade `high / critical` de produção: `npm audit --omit=dev --audit-level=high`.
2. **SonarQube no fluxo** — rodar o scanner a cada entrega e exigir o *Quality Gate* PASSED.
3. **Atualização automatizada** — Dependabot ou Renovate para PRs de patch/minor contínuos, evitando acúmulo de débito (foi o acúmulo que gerou as 49 iniciais).
4. **Imagem mínima** — manter o multi-stage build e o `.dockerignore` para que o *tooling* de desenvolvimento nunca chegue à imagem de produção.

Anexo — Comandos executados

```
# SCA (dependências)
npm audit                # grafo completo: 20 moderadas (0 high/critical)
npm audit --omit=dev     # só produção:    2 moderadas (0 high/critical)

# SAST (código) – SonarQube Community self-hosted via Docker
docker compose -f docker-compose.sonar.yml up -d
npm run test:cov         # gera coverage/lcov.info
docker run --rm --network host -e SONAR_HOST_URL=... -e SONAR_TOKEN=... \
  -v "$PWD:/usr/src" sonarsource/sonar-scanner-cli
```

Histórico: o estado inicial do MVP tinha 49 vulnerabilidades (10 altas). A migração NestJS 11 + bcrypt 6 reduziu para 20 (0 altas), todas o mesmo advisory moderado e não explorável de `js-yaml`. O SonarQube confirma 0 vulnerabilidades no código (Quality Gate PASSED).